

Báo cáo bài tập lớn LTNC - AuctionPro

Môn học: Lập trình nâng cao Dự án: AuctionPro - Online Auction System Nhóm: 4 thành viên Repository: <https://github.com/cecon123/online-auction-system>

1. Mục tiêu và phạm vi thực hiện

AuctionPro là hệ thống đấu giá trực tuyến dạng desktop, được xây dựng theo mô hình client-server để mô phỏng quy trình đăng bán sản phẩm, tham gia đấu giá, cập nhật giá theo thời gian thực và quản trị hệ thống. Mục tiêu của nhóm là hoàn thiện một ứng dụng Java có đủ các phần chính của một sản phẩm phần mềm: giao diện JavaFX, giao tiếp mạng bằng TCP socket, lưu trữ dữ liệu bền vững bằng SQLite, xử lý nghiệp vụ tập trung ở server, phân quyền người dùng và kiểm thử tự động cho các luồng quan trọng.

Phạm vi demo tập trung vào môi trường local, trong đó một server có thể phục vụ nhiều client JavaFX chạy đồng thời trên các terminal khác nhau. Hệ thống hỗ trợ ba vai trò chính: BIDDER tham gia đặt giá, SELLER tạo và quản lý phiên đấu giá, ADMIN quản trị người dùng và phiên đấu giá. Các chức năng nâng cao gồm ví điện tử, phong tỏa tiền khi đặt giá, auto-bid theo mức tối đa, broadcast sự kiện realtime và xử lý concurrent bidding khi nhiều người đặt giá cùng lúc.

Nhóm không đặt mục tiêu triển khai production deployment trên cloud hoặc thanh toán thật. Thay vào đó, hệ thống ưu tiên tính hoàn chỉnh trong phạm vi bài tập lớn: chạy được bằng file JAR, có dữ liệu mẫu để demo nhanh, có quy trình build/test bằng Maven và có CI/CD tự động tạo artifact release trên GitHub khi đánh tag phiên bản.

2. Kiến trúc tổng thể

```
JavaFX Client(s)
| JSON request/response over TCP socket
v
SocketServer
-> ClientHandler
-> RequestRouter
-> Auth / Auction / Bid / Wallet / Admin handlers
-> Service layer
-> DAO layer
-> SQLite database

NotificationService
-> BID_UPDATE / AUCTION_LIST_UPDATED / USER_LIST_UPDATED / SYSTEM_NOTIFICATION
-> subscribed clients
```

Dự án được chia thành ba module Maven chính. Module common chứa DTO, enum, model và protocol dùng chung để client và server thống nhất kiểu dữ liệu. Module server chịu trách nhiệm mở socket, quản lý kết nối, định tuyến request, kiểm tra phiên đăng nhập, xử lý nghiệp vụ, truy cập SQLite và phát sự kiện realtime. Module client triển khai giao diện JavaFX, controller, service phía client, cache trạng thái màn hình và SocketClient để trao đổi dữ liệu với server.

Luồng request tiêu chuẩn bắt đầu từ client tạo Request có requestId, type, payload và token phiên nếu cần xác thực. Request được gửi theo định dạng JSON qua TCP. Ở server, ClientHandler đọc từng message, RequestRouter chuyển request tới handler phù hợp, handler gọi service để validate và cập nhật dữ liệu. Kết quả trả về là Response có cùng requestId, giúp client ghép đúng phản hồi với thao tác đang chờ. Cách thiết kế này tách rõ giao diện, protocol và nghiệp vụ, đồng thời giúp kiểm thử service độc lập với UI.

Với realtime update, server không yêu cầu client polling liên tục. Sau khi một bid hợp lệ được ghi nhận hoặc trạng thái phiên đấu giá thay đổi, NotificationService gửi event tới các client đang kết nối. Client lắng nghe event nền và cập nhật lại danh sách, chi tiết phiên hoặc thông báo hệ thống. Hướng này phù hợp với bài toán đấu giá vì thông tin giá hiện tại phải được phản ánh nhanh cho nhiều người dùng.

3. Cấu trúc module và trách nhiệm

Module	Thành phần chính	Vai trò trong hệ thống
common	Model, DTO, enum, protocol, request/response	Là hợp đồng dữ liệu chung giữa client và server, giúp hai phía build độc lập nhưng vẫn thống nhất định dạng trao đổi
server	Socket server, router, handlers, services, DAO, database schema	Là nơi xử lý nghiệp vụ chính, xác thực, phân quyền, cập nhật database, quản lý transaction và phát realtime event

Module	Thành phần chính	Vai trò trong hệ thống
client	JavaFX views/controllers, client services, socket client	Hiển thị giao diện, nhận thao tác người dùng, gửi request tới server và cập nhật UI theo response/event
.github và docs	GitHub Actions, README, deployment guide, release guide, báo cáo	Hỗ trợ kiểm thử tự động, đóng gói artifact và cung cấp tài liệu nộp bài/chạy demo

Việc tách module giúp nhóm giảm phụ thuộc giữa các phần. Ví dụ, thay đổi giao diện JavaFX không bắt buộc sửa service phía server nếu protocol không đổi. Ngược lại, khi bổ sung validation ở server, client chỉ cần xử lý response lỗi tương ứng. Đây là cách tổ chức phù hợp với nhóm nhiều thành viên vì mỗi người có thể phụ trách một phần tương đối độc lập: UI, server service, database/DAO hoặc test/tài liệu.

Trong module server, nhóm dùng cách chia theo tầng: handler nhận request và kiểm tra dữ liệu đầu vào, service chứa nghiệp vụ, DAO thao tác SQLite. Với các nghiệp vụ cần tính nhất quán như đặt giá và ví điện tử, service là nơi điều phối transaction để đảm bảo nhiều bảng dữ liệu được cập nhật cùng nhau. Trong module client, controller không thao tác socket trực tiếp cho mọi chi tiết mà đi qua service phía client, giúp code UI dễ đọc hơn và giảm lặp logic gọi API.

4. Thiết kế dữ liệu và luồng nghiệp vụ chính

SQLite được dùng làm database nhưng để đơn giản hóa cài đặt và phù hợp với demo local. Các nhóm dữ liệu chính gồm người dùng, phiên đấu giá, lịch sử bid, ví điện tử, giao dịch ví và cấu hình auto-bid. Server là nguồn dữ liệu tin cậy duy nhất: client không tự quyết định kết quả nghiệp vụ mà chỉ gửi yêu cầu và hiển thị trạng thái được server xác nhận.

Luồng đăng nhập và phân quyền được xử lý ở server. Mật khẩu được hash bằng BCrypt trước khi lưu. Sau khi đăng nhập thành công, server cấp token phiên để client gửi kèm các request cần quyền truy cập. Router và handler kiểm tra role trước khi gọi service, ví dụ chỉ SELLER được tạo phiên đấu giá và chỉ ADMIN được khóa người dùng hoặc can thiệp phiên bất thường.

Luồng đặt giá được thiết kế để bảo vệ tính nhất quán. Khi bidder gửi bid, server kiểm tra phiên còn hoạt động, người đặt giá không phải chủ phiên, số tiền hợp lệ theo bước giá tối thiểu và ví còn đủ khả năng phong tỏa. Nếu bid mới vượt bid cũ, hệ thống hoàn lại phần tiền bị khóa của người trước đó, phong tỏa tiền của người đang thắng và ghi lịch sử giao dịch. Sau khi transaction thành công, server mới broadcast BID_UPDATE cho các client.

Auto-bid được xử lý cùng tầng service thay vì để client tự tăng giá. Người dùng đặt mức tối đa, server lưu rule theo từng phiên và tự quyết định bid kế tiếp dựa trên giá hiện tại, bước giá tối thiểu và giới hạn ví. Việc gom logic auto-bid ở server giúp tất cả client tuân theo cùng một quy tắc, tránh tình trạng mỗi client xử lý khác nhau khi có nhiều người tham gia.

5. Chức năng đã hoàn thành

Nhóm chức năng	Kết quả đạt được	Hướng giải quyết và lý do lựa chọn
Xác thực và phân quyền	Đăng ký, đăng nhập, role BIDDER/SELLER/ADMIN, khóa tài khoản	Dùng BCrypt, session token và kiểm tra quyền ở server để tránh tin tưởng dữ liệu từ client
Quản lý đấu giá	Seller tạo/sửa phiên, upload ảnh, bidder xem danh sách và chi tiết	Tách DTO/protocol dùng chung để UI thay đổi không ảnh hưởng trực tiếp tới DAO
Live bidding realtime	Nhiều client nhận cập nhật giá, trạng thái và thông báo	Dùng socket hai chiều và event listener, giảm độ trễ so với polling
Ví điện tử	Theo dõi balance, lockedBalance, nạp tiền demo, phong tỏa và hoàn tiền	Cập nhật ví trong transaction cùng bid để dữ liệu tiền và lịch sử bid nhất quán
Concurrent bidding	Xử lý nhiều request đặt giá vào cùng một phiên	Khóa theo auctionId, kiểm tra lại giá hiện tại trong transaction và dùng requestId để giảm rủi ro xử lý lặp
Auto-bid	Đặt mức tối đa và tự tăng giá theo bước tối thiểu	Đặt logic ở server để đảm bảo công bằng và thống nhất với validation của bid thủ công
Admin	Quản lý user, xem và xử lý phiên đấu giá bất thường	Tách nhóm handler/service admin, chỉ role ADMIN được truy cập

Nhóm chức năng	Kết quả đạt được	Hướng giải quyết và lý do lựa chọn
Kiểm thử và CI/CD	Unit, integration, socket test, GitHub Actions build/test/release	Dùng JUnit 5/Mockito, SQLite test thật và workflow tự build JAR khi tạo tag

Ngoài các chức năng chính, hệ thống cũng có các chi tiết hỗ trợ demo như dữ liệu mẫu, hướng dẫn tài khoản demo trong README, tài liệu deployment và quy trình release. Những phần này giúp giảng viên hoặc người chấm có thể build, chạy server/client và kiểm tra chức năng mà không cần cấu hình phức tạp.

6. Điểm kỹ thuật quan trọng

Vấn đề có rủi ro cao nhất trong hệ thống là concurrent bidding. Nếu hai client cùng đặt giá gần như đồng thời, server phải đảm bảo chỉ một kết quả cuối cùng được chấp nhận theo dữ liệu mới nhất. Nhóm xử lý bằng cách gom toàn bộ logic đặt giá vào server service, khóa theo từng auctionId và thực hiện kiểm tra/cập nhật trong database transaction. Nhờ vậy, các phiên đấu giá khác nhau vẫn có thể xử lý song song, còn các bid trên cùng một phiên được tuần tự hóa để tránh race condition.

Realtime update cũng là phần quan trọng vì nó quyết định trải nghiệm demo. Server phát event sau khi dữ liệu đã được commit, không phát trước transaction. Client cập nhật UI theo event nhận được nhưng vẫn coi server là nguồn dữ liệu chính. Khi cần đồng bộ lại toàn bộ màn hình, client có thể gửi request lấy danh sách hoặc chi tiết phiên để tránh lệch trạng thái sau khi mất kết nối.

Với ví điện tử, nhóm chọn mô hình balance và lockedBalance. Khi bidder đang thắng một phiên, số tiền tương ứng được khóa để tránh dùng cùng một khoản tiền cho nhiều phiên khác nhau. Nếu bidder bị vượt giá, tiền bị khóa được hoàn lại. Cách làm này đơn giản hơn so với tích hợp thanh toán thật nhưng vẫn thể hiện được nghiệp vụ tài chính cơ bản của hệ thống đấu giá.

Về đóng gói, server và client đều có executable fat JAR để chạy trực tiếp bằng java -jar. Riêng client JavaFX cần đóng gói dependency theo nền tảng, vì JavaFX có native runtime khác nhau trên Linux, Windows và macOS. CI/CD vì vậy tạo artifact client riêng cho từng hệ điều hành trong GitHub Release, còn server JAR dùng chung.

Một điểm đáng chú ý khác là cách xử lý lỗi. Các lỗi như sai thông tin đăng nhập, tài khoản bị khóa, phiên đấu giá đã kết thúc, bid thấp hơn giá tối thiểu hoặc ví không đủ tiền đều được trả về dưới dạng response lỗi có thông điệp rõ ràng. Client hiển thị lỗi này ở đúng màn hình thao tác, còn server vẫn ghi log để phục vụ debug. Cách xử lý này giúp demo dễ theo dõi và tránh trường hợp lỗi nghiệp vụ làm client treo hoặc mất kết nối.

7. Kiểm thử, build và demo

Project dùng Maven multi-module với Java 21. Lệnh kiểm thử đầy đủ là:

```
mvn clean verify
```

Lệnh đóng gói artifact:

```
mvn clean package
```

Sau khi build, các file JAR chính nằm tại:

- server/target/auction-server.jar
- client/target/auction-client.jar

Thứ tự chạy demo:

```
java -jar server/target/auction-server.jar
java -jar client/target/auction-client.jar
```

Để demo nhiều client, mở thêm terminal và chạy lại client JAR. Server mặc định dùng socket port 8080, asset port 8081, SQLite database auction.db và thư mục upload uploads/. Kịch bản demo phù hợp là đăng nhập seller để tạo phiên đấu giá, mở hai client bidder để đặt giá cạnh tranh, quan sát realtime update, sau đó dùng admin kiểm tra trạng thái người dùng hoặc phiên đấu giá.

CI/CD trên GitHub Actions chạy build và test khi có push hoặc pull request vào dev/main. Khi tạo tag dạng v*, workflow sẽ build release artifact cho server, client Linux, client Windows, client macOS và đính kèm báo cáo PDF vào GitHub Release. Quy trình này giúp lần nộp cuối chỉ cần merge source ổn định vào main, tạo tag phiên bản và kiểm tra release tự động.

Kịch bản demo đề xuất gồm bốn bước. Bước một là chạy server bằng JAR để xác nhận socket port và database được khởi tạo. Bước hai là mở hai hoặc ba client, đăng nhập bằng các vai trò khác nhau. Bước ba là seller tạo phiên đấu giá, hai bidder đặt giá liên tiếp và quan sát realtime update trên các cửa sổ client. Bước bốn là demo một tình huống kỹ thuật: hai bidder đặt giá gần nhau để thể hiện concurrent bidding, hoặc bật auto-bid để server tự đưa ra giá kế tiếp trong giới hạn tối đa.

8. Hạn chế và hướng phát triển

Trong phạm vi bài tập lớn, hệ thống ưu tiên chạy ổn định trên môi trường local nên vẫn còn một số giới hạn. Database SQLite phù hợp cho demo nhưng chưa phải lựa chọn tối ưu nếu triển khai nhiều server hoặc số lượng người dùng lớn. Cơ chế upload ảnh hiện phục vụ local asset server, chưa dùng object storage. Ví điện tử là mô phỏng nghiệp vụ, chưa tích hợp cổng thanh toán thật. Ngoài ra, kiểm thử UI JavaFX mới dừng ở mức vừa đủ cho CI, chưa bao phủ toàn bộ thao tác giao diện như một bộ end-to-end test hoàn chỉnh.

Nếu tiếp tục phát triển, hệ thống có thể chuyển database sang PostgreSQL/MySQL, bổ sung refresh token, phân trang/ lọc nâng cao, lưu file ảnh trên object storage, thông báo qua email và kiểm thử UI tự động sâu hơn. Các phần này không làm thay đổi kiến trúc chính, vì server đã là nơi tập trung nghiệp vụ và client giao tiếp qua protocol rõ ràng.

9. Kết luận

AuctionPro hoàn thành các yêu cầu cốt lõi của bài tập lớn LTNC: có kiến trúc client-server rõ ràng, giao diện desktop, lưu trữ dữ liệu, xác thực, phân quyền, ví điện tử, đấu giá realtime, xử lý đồng thời, kiểm thử và đóng gói chạy bằng JAR. Điểm mạnh của hệ thống là server giữ toàn bộ nghiệp vụ quan trọng, nhờ đó các client chỉ đóng vai trò giao diện và không thể tự quyết định trạng thái đấu giá.

Trong phạm vi môn học, nhóm ưu tiên sự ổn định khi demo local và khả năng kiểm tra lại bằng CI/CD, nên các thành phần được giữ đủ đơn giản để dễ build, dễ chạy và dễ đánh giá. Báo cáo, README và release artifact được đặt trực tiếp trong repository để người chấm có thể truy vết từ tài liệu tới source code, từ source code tới file JAR và từ file JAR tới kịch bản demo thực tế.